A cest proiect implementează un procesor MIPS32 single-cycle

Sunt implementate instrucțiuni aritmetice, logice, de memorie și de control.

Execuția e realizată secvențial, fiecare instrucțiune fiind finalizată într-un ciclu de ceas.

Setul de instrucțiuni pentru MIPS 32

Tip R	Adunare	add
	Scădere	sub
	Deplasare logică la stânga (shift left logical)	sll
	Deplasare la dreapta (shift right logical)	srl
	Operație logică ȘI	and
	Operație logică SAU	or
	XOR - bit wise exclusive - OR	xor
	Set on less than	SLT
Tip I	Adunare cu imediat (add immediate)	addi
	Încărcare cuvânt din memorie (load word)	lw
	Stocare cuvânt în memorie (store word)	sw
	Salt în caz de egalitate	beq
	Branch on Greater than or Equal to zero	BGEZ
	Branch on Greater Than Zero	BGTZ
Tip J	Salt necondiționat (jump)	j

1. Sintaxa în assembler
2. Descriere RTL abstract
3. Formatul pe biți în cod mașină cu valoarea câmpului opcode și funcție (unde este cazul)
4. Un exemplu concret în ASM și codificat pe biți în cod mașină.
   ex.: add $3 $1 $2 -> 6'b00000_00001_00010_00011_00000 (restul biților până la 32)

5 Diagrama de procesare (după model curs 3)

ADD - ADDition

Descriere	Adună registrele $s și $t. Stochează rez în $d
RTL	$d ← $s + $t ; PC ← PC + 4
Sintaxă	add $d, $s, $t
Format	000000 sssss ttttt ddddd 00000 000000

SUB - Subtraction

Descriere	Scade registrul $t din $s. Stochează rezultatul în $d
RTL	$ ← $s - $t ; PC ← PC + 4
Sintaxă	sub $d, $s, $t
Format	000000 sssss ttttt ddddd 00000 100001

SLL - Shift-Left Logical

Descriere	Deplasare logică la stânga pt. registrul $t cu h poziții. Stochează rezultatul în registrul $d
RTL	$d ← $t << h ; PC ← PC + 4
Sintaxă	sll $d, $t, h
Format	000000 00000 ttttt ddddd hhhhh 000010

SRL - Shift - Right Logical

Descriere	Deplasare logică la dreapta pentru registrul $t cu h poziții. Stochează rezultat $d
RTL	$d ← $t >> h ; PC ← PC + 4
Sintaxă	srl $d, $t, h
Format	000000 00000 ttttt ddddd hhhhh 000011

AND - bitwise AND

Descriere	Și între registrele $s și $t. Stochează rezultatul în registrul $d
RTL	$d ← $s & $t ; PC ← PC + 4
Sintaxă	and $d, $s, $t
Format	000000 sssss ttttt ddddd 00000 100100

OR - bitwise OR

Descriere	SAU între regiștrele $s și $t. Stochează rezultatul în registrul $d.
RTL	$d ← $s | $t ; PC ← PC + 4
Sintaxă	or $d, $s, $t
Format	000000 sssss ttttt ddddd 00000 000101

XOR - bitwise exclusive-OR

Descriere	XOR între regiștrele $s și $t. Stochează rez. în registrul $d
RTL	$d ← $s ⊕ $t ; PC ← PC + 4
Sintaxă	xor $d, $s, $t
Format	000000 sssss ttttt ddddd 00000 000110

SLT - Set on Less Than (signed)

Descriere	Dacă registrul $s < $t, atunci registrul $d devine 1, altfel devine 0
RTL	PC ← PC + 4 ; if $s < $t then $d ← 1 else $d ← 0
Sintaxă	slt $d, $s, $t
Format	000000 sssss ttttt ddddd 00000 000111

ADDI - ADD Immediate

Descriere	Adună registrul $s cu valoarea imm. Stochează rez. în $t
RTL	$t ← $s + SE(imm) ; PC ← PC + 4 ;
Sintaxă	addi $t, $s, imm
Format	111111 sssss ttttt iiiiiiiiiiiiiiii

LW - Load Word

Descriere	Încarcă un cuvânt de memorie în registrul $t
RTL	$t ← MEM[$s + SE(offset)] ; PC ← PC + 4 ;
Sintaxă	lw $t, offset($s)
Format	111110 sssss ttttt 0000000000000000

SW - Store Word

Descriere	Stochează într-un cuvânt de memorie valoarea registrului $t
RTL	MEM[$s + SE(offset)] ← $t ; PC ← PC + 4
Sintaxă	sw $t, offset($s)
Format	111101 sssss ttttt 0000000000000000

BEQ - Branch on Equal

Descriere	Realizează salt condiționat dacă regiștrele $s și $t sunt egale
RTL	if $s = $t then PC ← (PC + 4) + (SE(offset) << 2) else PC ← PC + 4
Sintaxă	beq $s, $t, offset
Format	111100 sssss ttttt 0000000000000000

BGEZ - Branch on Greater or Equal to Zero

Descriere	Realizează salt condiționat dacă registrul $s este pozitiv
RTL	if $s ≥ 0 then PC ← (PC + 4) + (SE(offset) << 2) else PC ← PC + 4 ;
Sintaxă	bgez $s, offset
Format	111011 sssss 00000 0000000000000000

BGTZ - Branch on Greater Than Zero

Descriere	Realizează salt condiționat dacă registrul $s este strict pozitiv
RTL	if $s > 0 then PC ← (PC + 4) + (SE(offset) << 2) else PC ← PC + 4 ;
Sintaxă	bgtz $s, offset
Format	111010 sssss 00000 0000000000000000

J - Jump

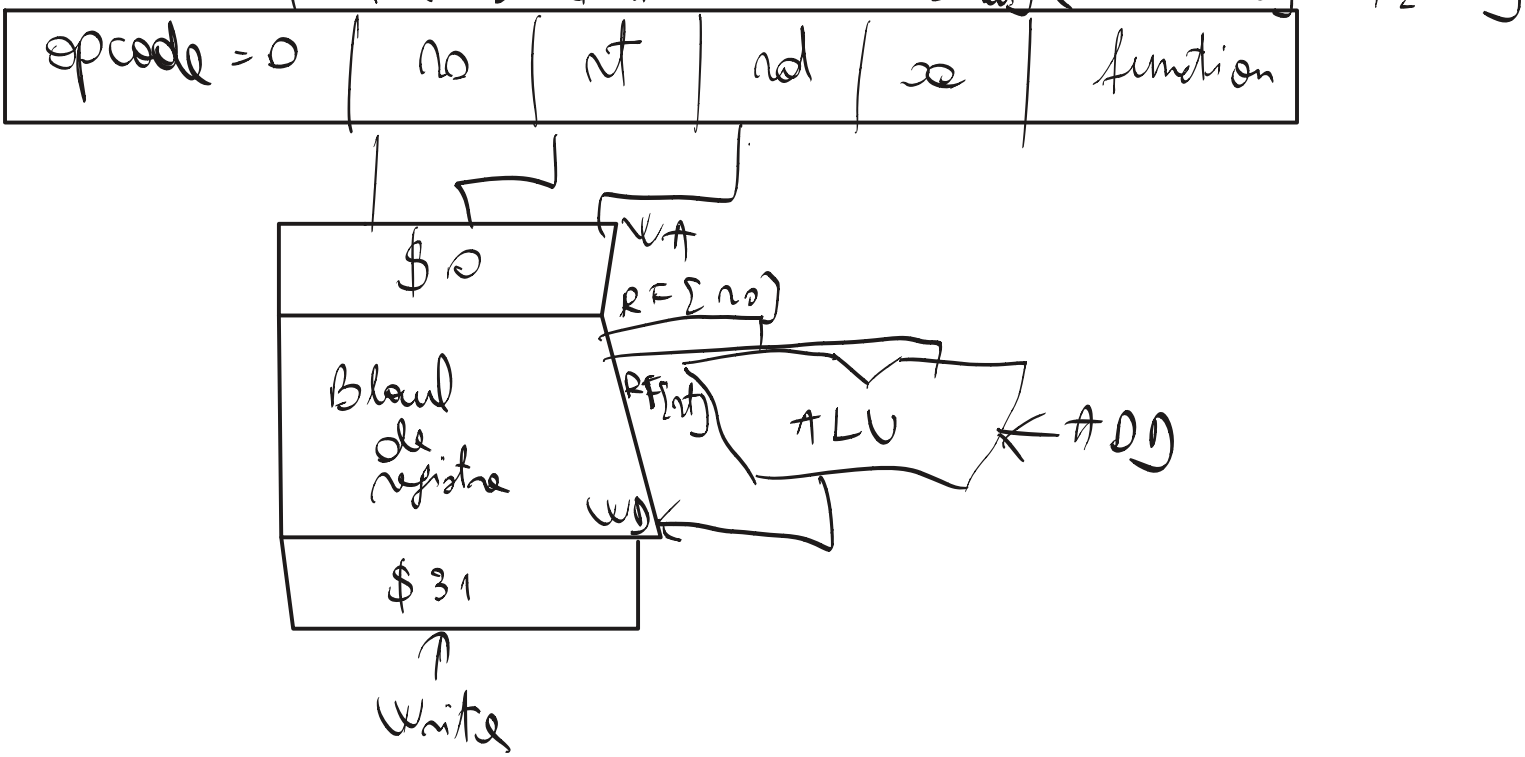
Descriere	Realizează salt necondiționat la adresa dorită.
RTL	PC ← (PC + 4)[31:28] || (addr << 2) ;
Sintaxă	j addr
Format	111001 00000000000000000000000000

Exemple cod mașină / ASM

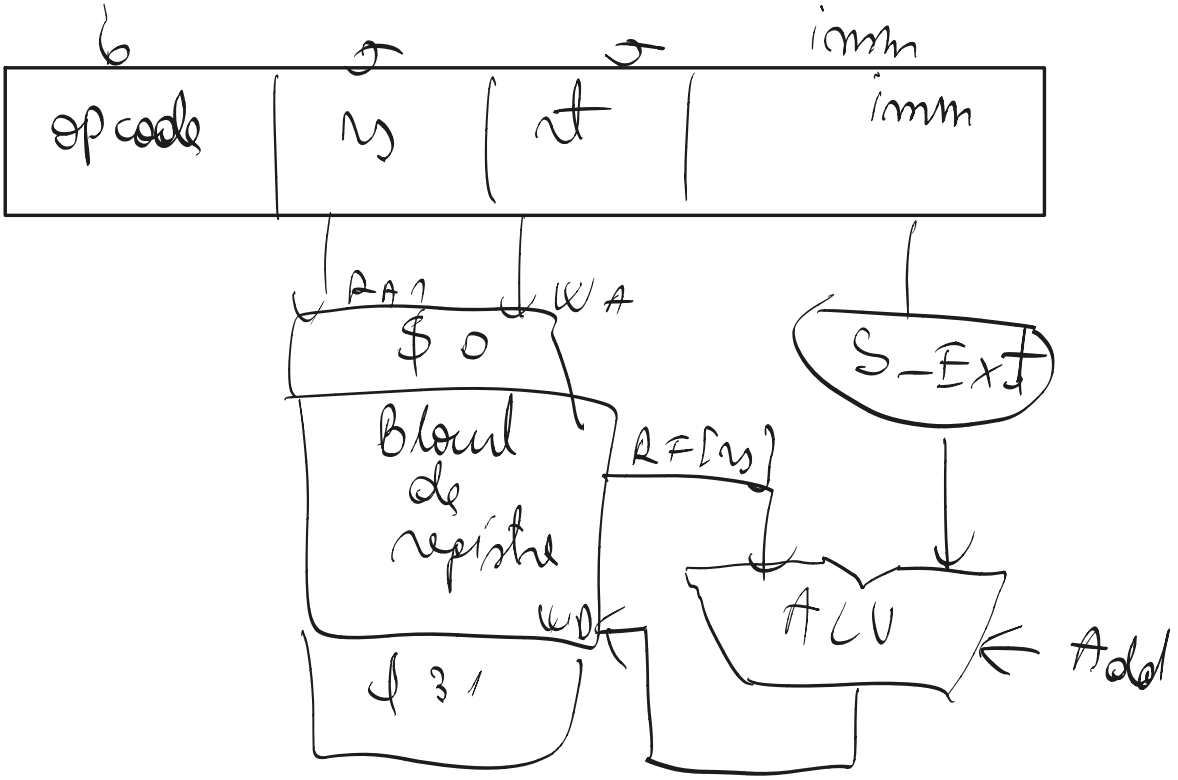
ADD $1, $2, $3	B"000000_00010_00011_00001_00000_100000"	X
SUB $1, $2, $3	B"000000_00010_00011_00001_00000_100001"	X
SLL $1, $2, 1	B"000000_00000_00010_00001_00001_000010"	X
SRL $1, $2, 2	B"000000_00000_00010_00001_00010_000011"	X
AND $1, $2, $3	B"000000_00010_00011_00001_00000_000100"	X
OR $1, $2 $3	B"000000_00010_00011_00001_00000_000101"	X
XOR $1, $2, $3	B"000000_00010_00011_00001_00000_000110"	X
SLT $1, $2, $3	B"000000_00010_00011_00001_00000_000111"	X
ADDi $1, $2, 1	B"111111_00010_00001_0000000000000001"	X
LW $1, 2($2)	B"111110_00010_00001_0000000000000010"	X
SW $1, 3($2)	B"111101_00010_00001_0000000000000011"	X
BEQ $1, $2, 4	B"111100_00010_00001_0000000000000100"	X
BGEZ $1, 5	B"111011_00001_00000_0000000000000101"	X
BGTZ $1, 6	B"111010_00001_00000_0000000000000110"	X
j 1	B"111001000000000000000000000001"	X

FUNCTIONALITATI

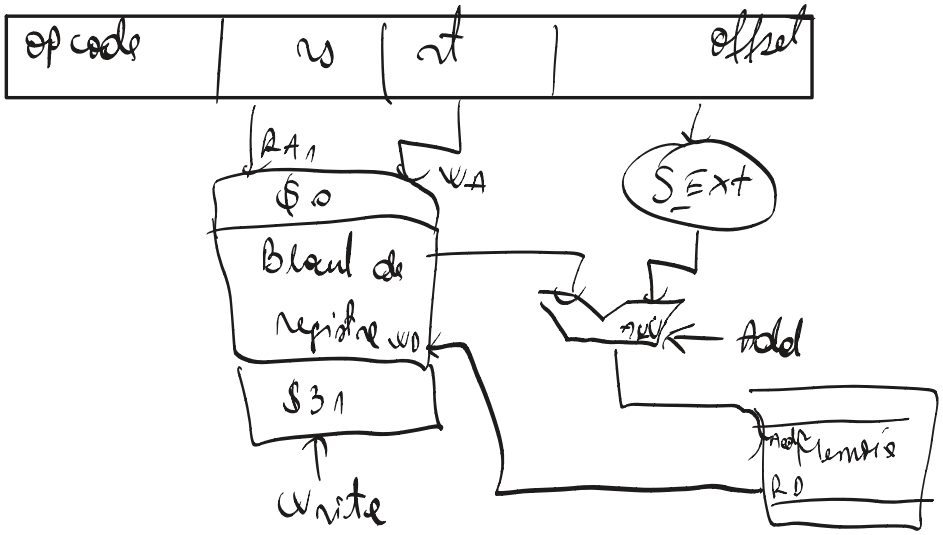
add $rd, $rs, $rt RF[rd] ← RF[rs] + RF[rt]



addi $rt, $rs, imm RF[rt] ← RF[rs] + S_Ext(imm)

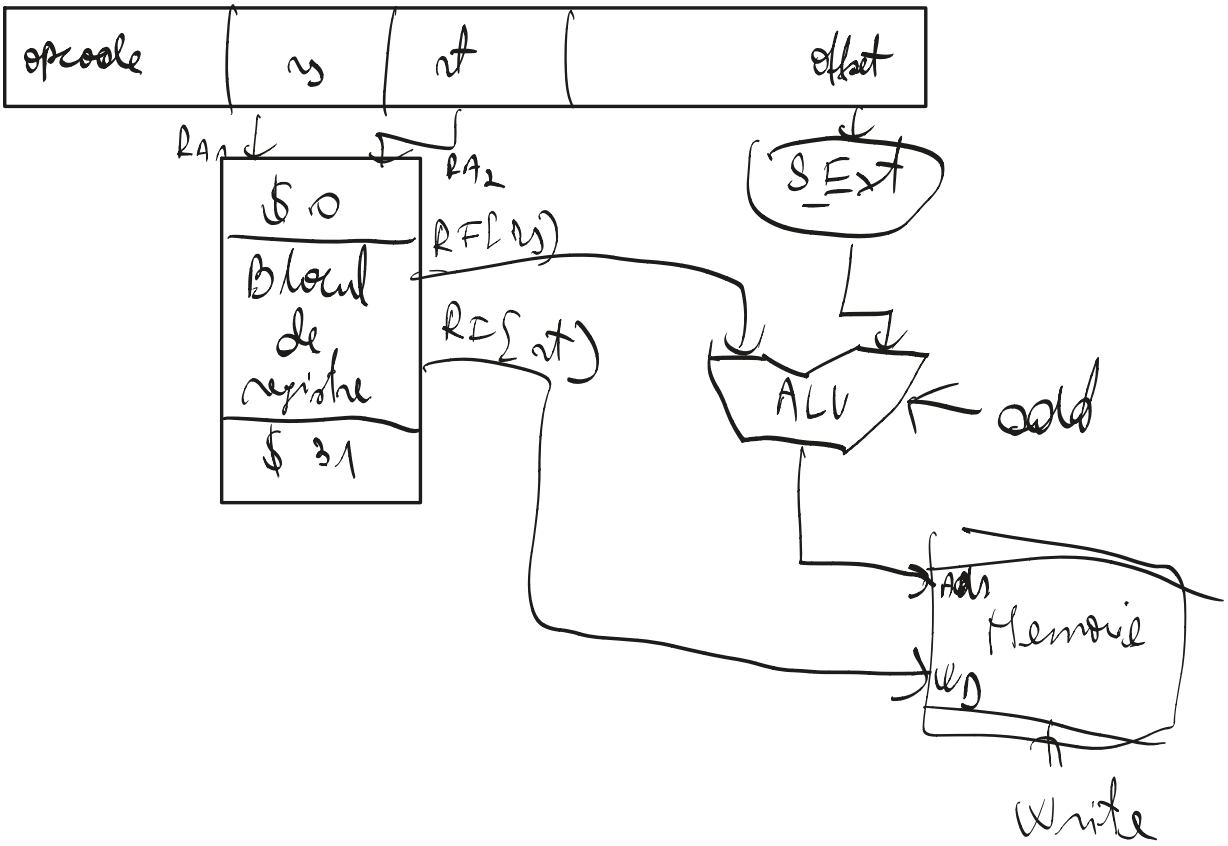


lw $rt, offset($rs) RF[rt] ← M[RF[rs] + S_Ext(offset)]



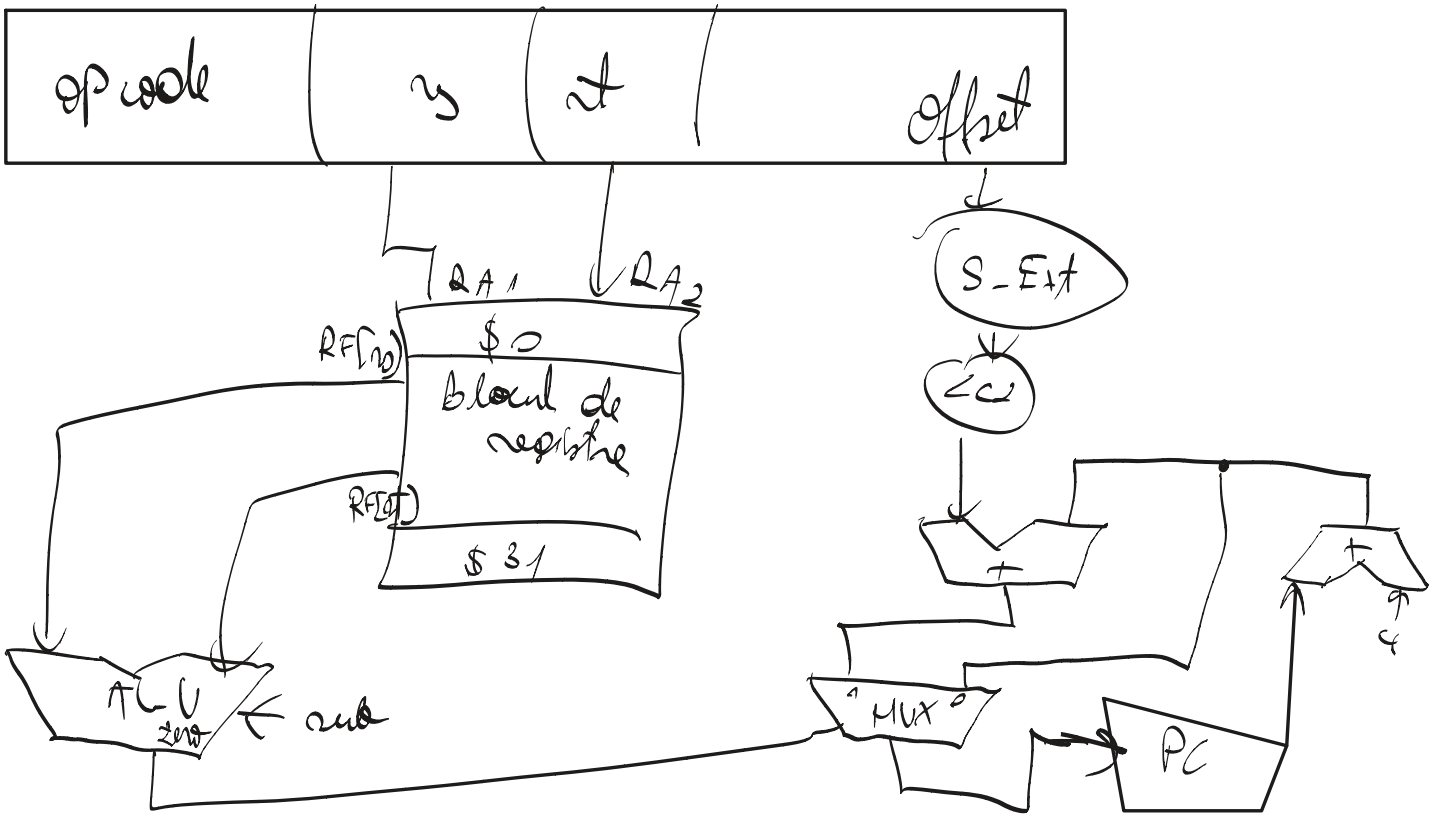
sw $rt, offset($rs)

MEM[RF[rs] + S_ext(offset)] ← RF[rt]



beq $rs $rt, offset

if RF[rs] = RF[rt] then PC ← (PC+4) + S_Ext(offset)<<2 else PC ← PC+4

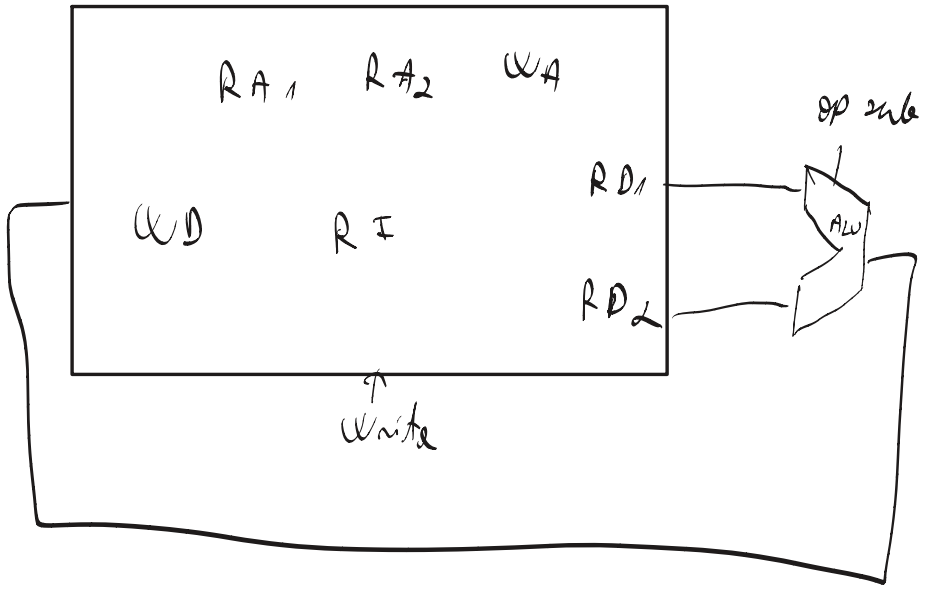


j adress $PC \leftarrow (PC+4)[31:28] \,||\, adress \,||\, 00$

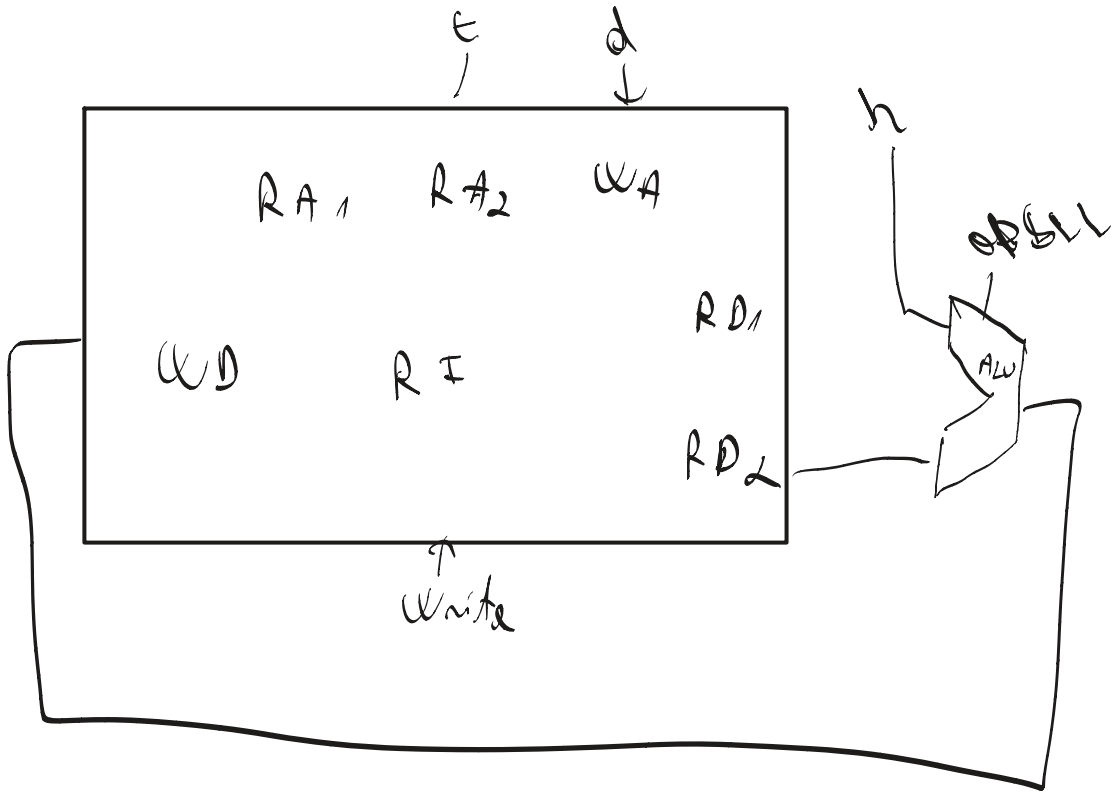
6 26

$(PC+4)_{31:28}$

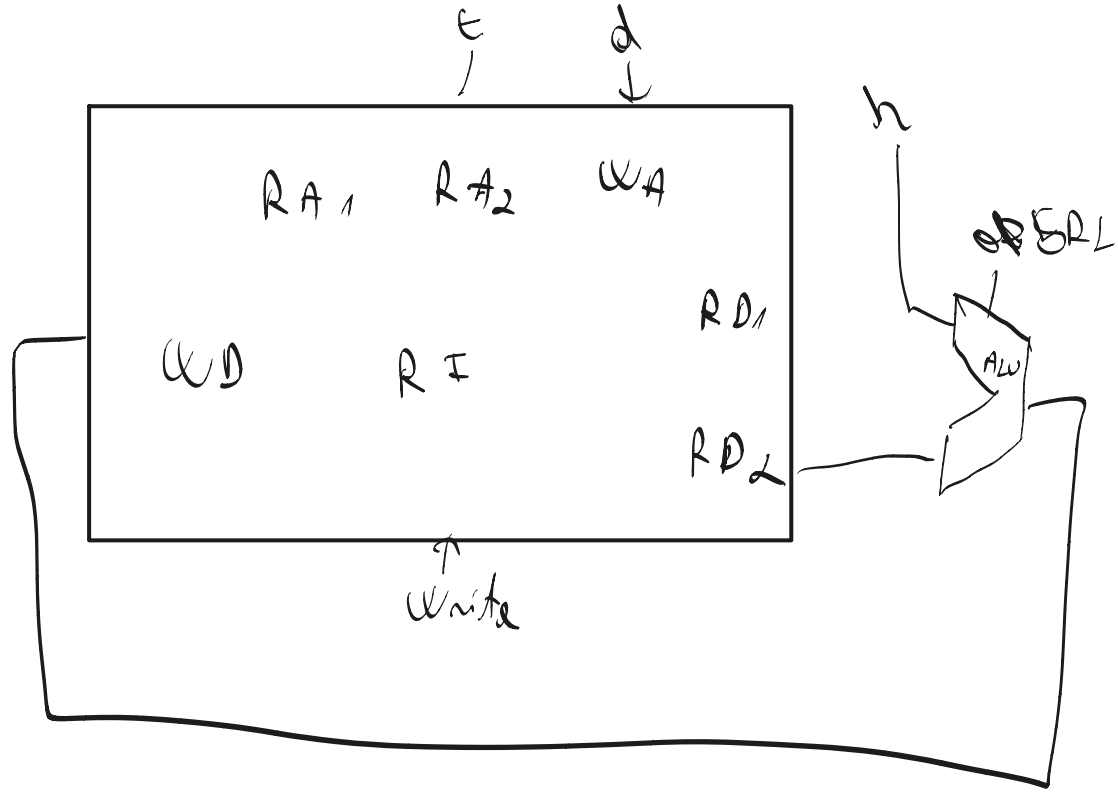
sub $d, $s, $t $d \leftarrow $s - $t, $PC \leftarrow PC+4$

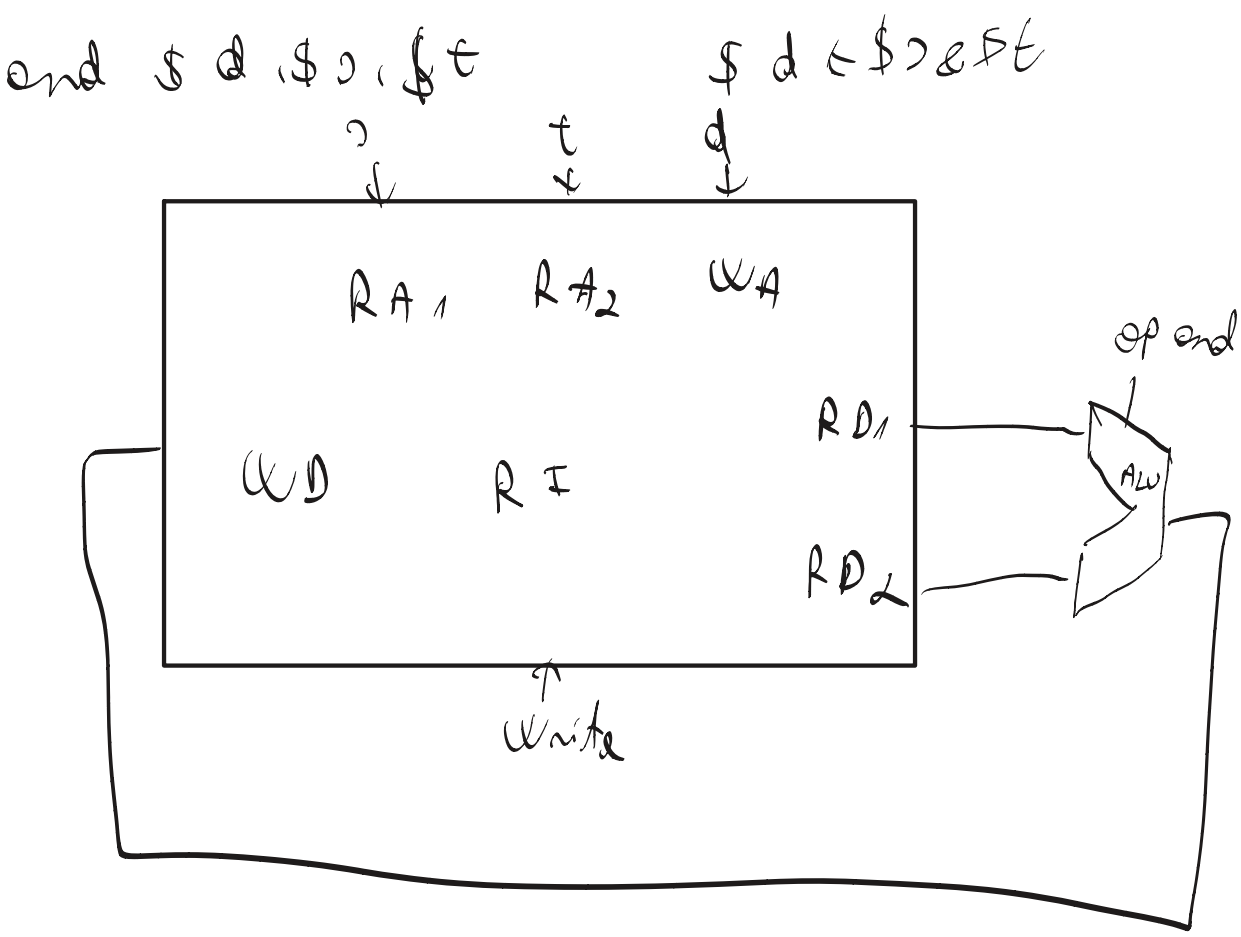


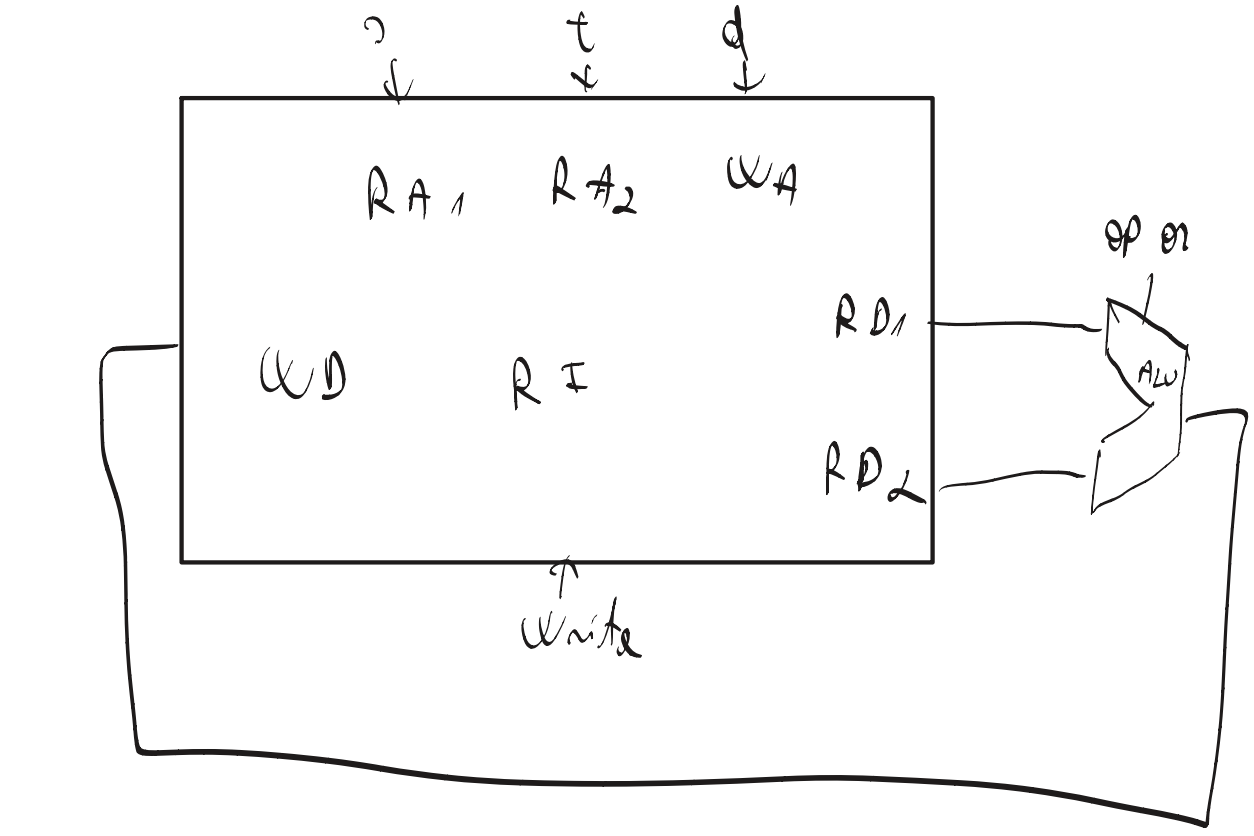
addi $d, $s, h $d ← $s + ch ; PC ← PC+4



srl $d, $s, h $d ← $s >> h ; PC ← PC+4

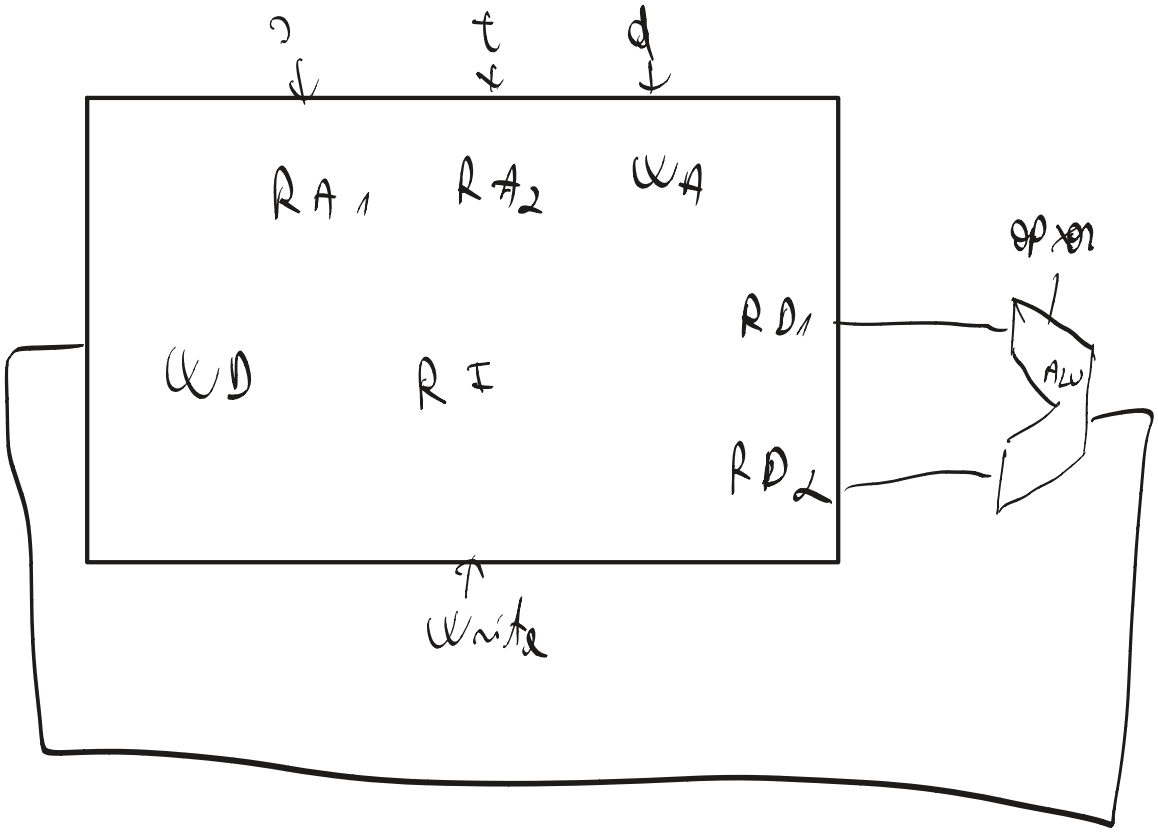



RA1
RA2
WA
WD
RI
RD1
RD2
ALU
Write


RA1
RA2
WA
WD
RI
RD1
RD2
ALU
Write

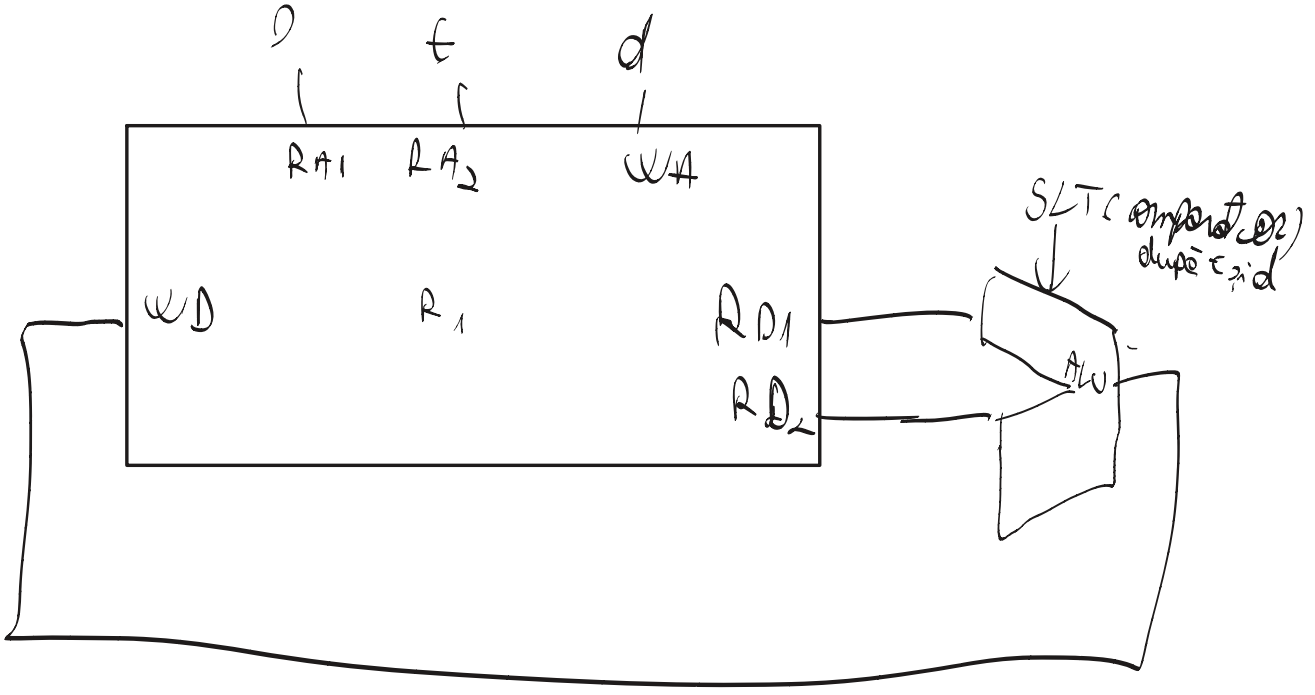
xor $d, $s, $t

$d ← $s ⊕ $t



SLT $d, $s, $t

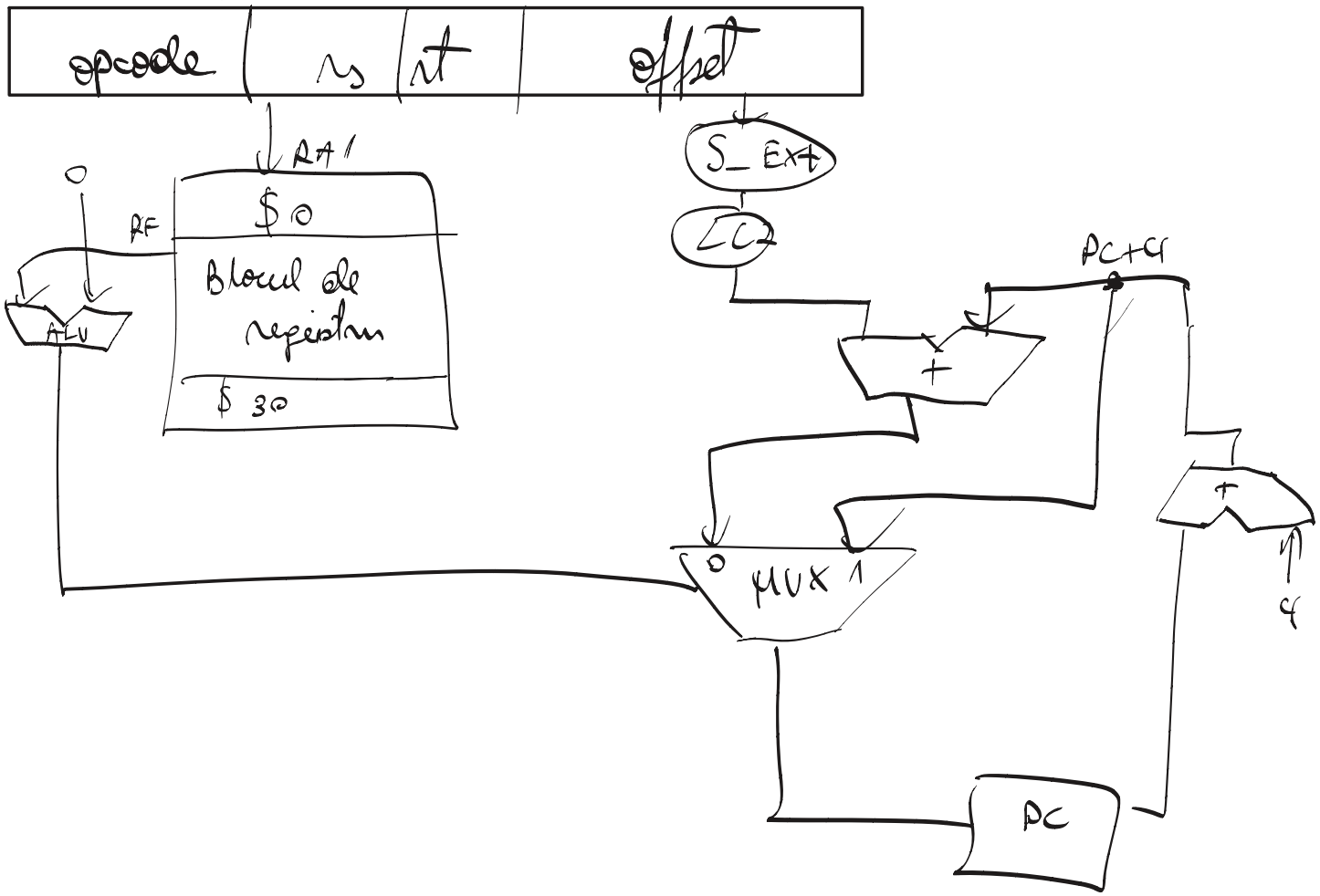
PC ← PC+4 ; if $s < $t then $d ← 1 else $d ← 0



$d ← 0 ou 1

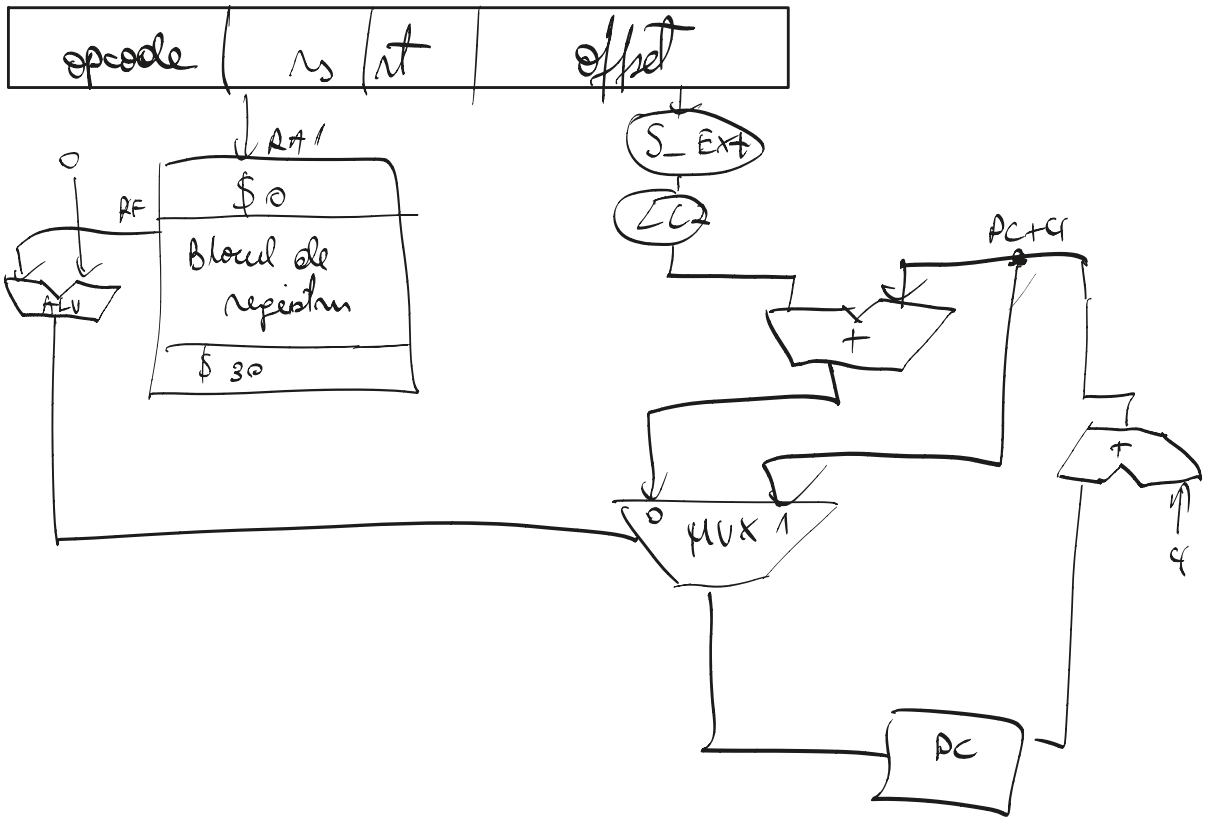
if RF[rs] >= 0 then PC ← (PC+4) + S_Ext(offset) << 2 else PC ← PC+4

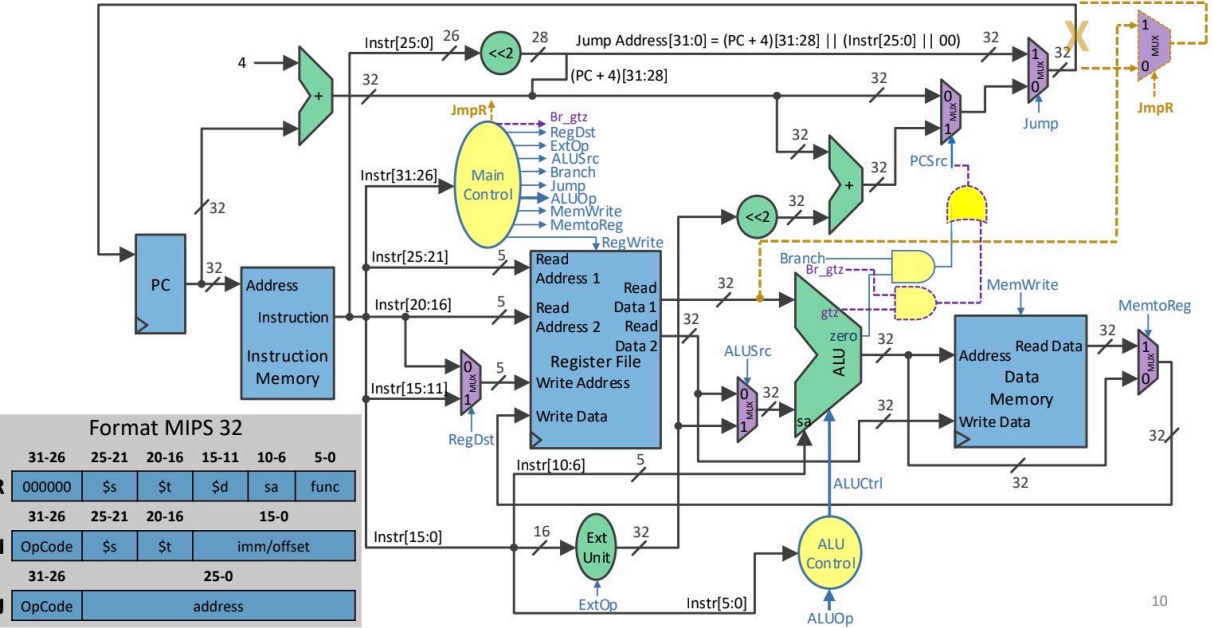
BGTZ $rs, offset



if RF[rs] >= 0 then
PC ← (PC+4) + S_Ext(offset) << 2 else PC ← PC+4

BGEZ $rs, offset




Instr[25:0]
26
<<2
28
Jump Address[31:0] = (PC + 4)[31:28] || (Instr[25:0] || 00)
32
4
+
32
(PC + 4)[31:28]
32
JmpR
Br_gtz
RegDst
ExtOp
ALUSrc
Branch
Jump
ALUOp
MemWrite
MemtoReg
Main Control
Instr[31:26]
RegWrite
Jump
PCSrc
32
PC
Address
Instruction
Instruction Memory
Instr[25:21]
Instr[20:16]
Instr[15:11]
5
Read Address 1
Read Address 2
Write Address
Write Data
Read Data 1
Read Data 2
Register File
RegDst
Branch
Br_gtz
gtz
zero
ALU
sa
ALUSrc
MemWrite
MemtoReg
Address
Read Data
Data Memory
Write Data
Instr[10:6]
ALUCtrl
Instr[15:0]
16
Ext Unit
ExtOp
ALU Control
Instr[5:0]
ALUOp
Format MIPS 32
31-26
25-21
20-16
15-11
10-6
5-0
R
000000
$s
$t
$d
sa
func
31-26
25-21
20-16
15-0
I
OpCode
$s
$t
imm/offset
31-26
25-0
J
OpCode
address

Semnale de control MIPS32

Semnale de Branch opționale: ? ∈ {gez, ne, gtz}, se va înlocui ? cu o valoare din paranteză, dacă e cazul

Tipuri de operații care se pun în paranteză la ALUOp si ALUCtrl:

(+), (-), (&), (|), (^), (<<*l*), (<<*lv*), (>>*l*), (>>*a*), (<)

Semnificații: & - AND, | - OR, ^ - *XOR, l - logic, a - aritmetic, v - cu variabilă*

Instrucțiune	Opcode *Instr[31-26]*	Reg Dst	ExtOp	ALUSrc	Branch	B_GEZ	Jump	B_GTZ	Mem Write	Memto Reg	Reg Write	ALUOp[1:0]	function *Instr[5-0]*	ALUCtrl[2:0]
ADD	000000	1	0	0	0	0	0	0	0	0	1	10	100000	000
SUB	000000	1	0	0	0	0	0	0	0	0	1	10	100010	001
SLL	000000	1	0	0	0	0	0	0	0	0	1	10	000000	100
SRL	000000	1	0	0	0	0	0	0	0	0	1	10	000010	110
AND	000000	1	0	0	0	0	0	0	0	0	1	10	100100	010
OR	000000	1	0	0	0	0	0	0	0	0	1	10	100101	011
XOR	000000	1	0	0	0	0	0	0	0	0	1	10	100110	100
SLT	000000	1	0	0	0	0	0	0	0	0	1	10	101010	111
ADDi	001000	0	1	1	0	0	0	0	0	0	1	00	XXXXXX	000
LW	100011	0	1	1	0	0	0	0	0	1	1	00	XXXXXX	000
SW	101011	X	1	1	0	0	0	0	1	X	0	00	XXXXXX	000
BEQ	000100	X	1	1	1	0	0	0	0	X	0	01	XXXXXX	001
BGEZ	000001	X	1	1	0	1	0	0	0	X	0	01	XXXXXX	001
BGTZ	000111	X	1	1	0	0	0	1	0	X	0	01	XXXXXX	001
J	000010	X	X	0	0	0	1	0	0	X	0	XX	XXXXXX	XXX

URL: https://drive.google.com/file/d/1SI7x2Gp_2m3SEkwnXuGt4ns4voYzpGBH/view?usp=sharing

Pe data viitoare de implementat problema 13. ANEXA 7
Am implementat

---

programul inserto secvențial instrucțiuni MIPS (aritmetice, logice și de memorie) într-un executabil este controlată de

15. Se citește o valoare X și o adresă de început A din memorie
- se parcurge memoria începând de la A până la prima valoare 0
- se numără câte valori sunt mai mici decât X, iar rezultatul se scrie la adresa 0
Cum gândesc?

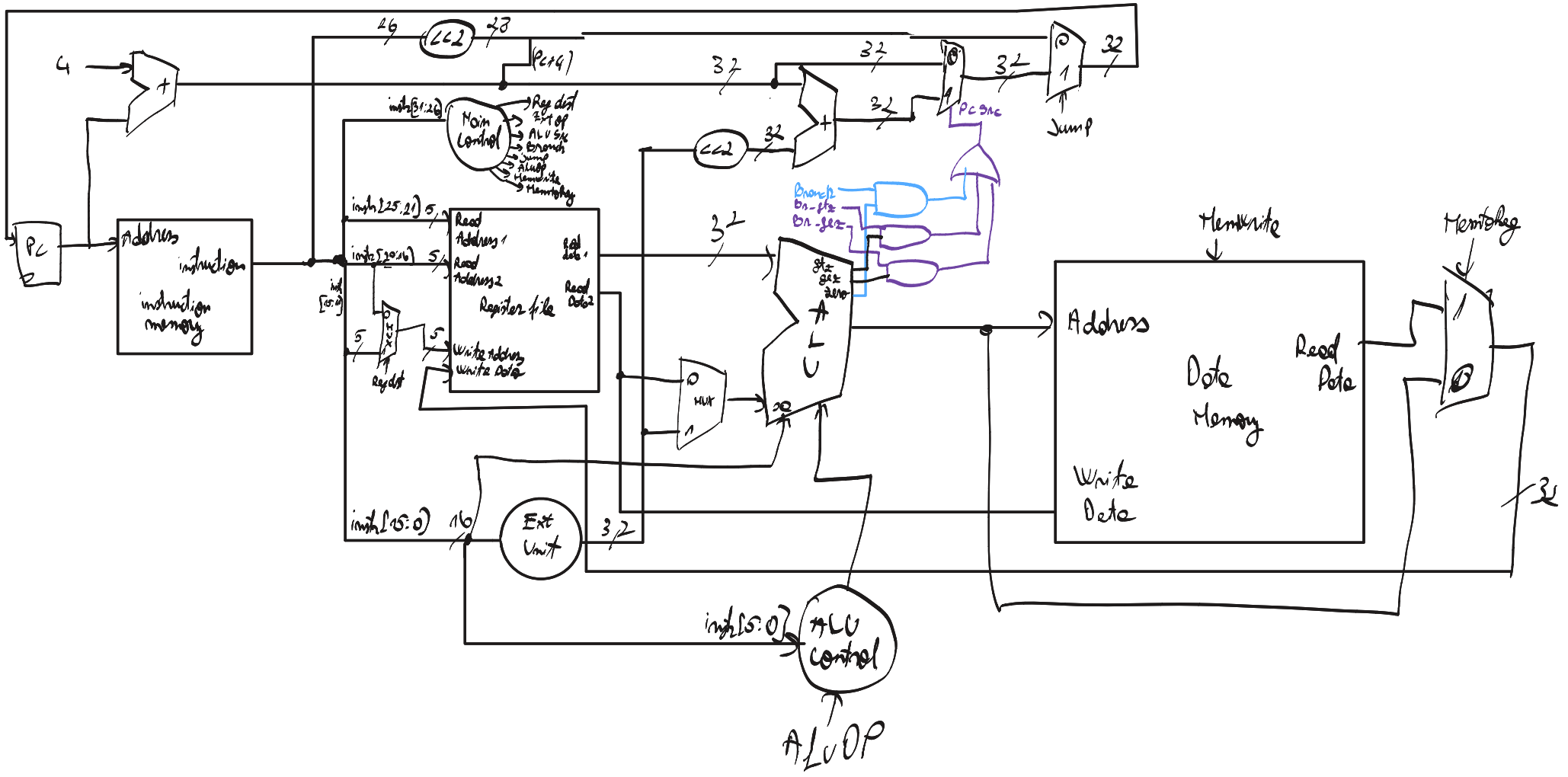
```
X = MEM[0]
A = MEM[4]
count = 0
while MEM[A] != 0 do
    if MEM[A] < X then
        count = count + 1
    end if
    A = A + 4
end while.
```

Pe scurt
Ampus A = A+4 pentru că lucrăm cu valori de biți, adică 4 octeți fiecare
Deci următorul element din șir e la adresa următoare din 4 în 4.

Execuția completă a algoritmului are nevoie de ALU, control, memorie, un drum Fetch

MEM(0) = 10 ; X
MEM[4] = 12 ; A
MEM[12] = 7
MEM[16] = 15
MEM[20] = 3
MEM[24] = 11
MEM[28] = 0

7 < 10 → da ✓
15 < 10 → nu
3 < 10 → da ✓
11 < 10 → nu
0 → stop
Rezultatul este 2
MEM(0) = 2


4
+
(PC+4)
32
Main Control
Reg dst
Ext op
ALU src
Branch
Jump
MemRead
MemWrite
MemtoReg
Jump
PC src
PC
Address
instruction
instruction memory
Read Address 1
Read Address 2
Register file
Read Data 1
Read Data 2
Write Address
Write Data
RegWrite
Ext
MUX
ALU
Branch
Br_gtz
Br_gez
zero
MemWrite
MemtoReg
Address
Data Memory
Read Data
Write Data
Ext Unit
ALU control
ALUOP

Trasarea execuției programului de test pentru MIPS32

Valorile se completează în hexazecimal așa cum trebuie să apară pe SSD. Succesiunea pașilor reprezintă ordinea de execuție în timp la apăsarea butonului ENable. **Pasul 0 corespunde stării inițiale a circuitului (PC = 0), iar pasul *N* caracterizează starea după apăsarea de *N* ori a butonului ENable**. Inițial registrele vor avea valoarea 0 (care se atribuie automat în lipsa unei inițializări explicite a RF), iar memoria de date RAM poate fi inițializată cu valori dorite. Tabelul se completează pentru tot programul sau, dacă are buclă, **cel puțin** până se execută complet prima iterație și prima instrucțiune de la începutul iterației 2. *Buclă = revenirea execuției la o instrucțiune care a mai fost executată anterior.*

Pas	SW(7:5)	"000"	"001"	"010"	"011"	"100"	"101"	"110"	"111"	De completat numai pentru instrucțiuni de salt	
	Instr (*în asamblare*)	Instr (*hexa*)	PC+4	RD1	RD2	Ext_Imm	ALURes	MemData	WD	BranchAddr	JumpAddr
0	ADD $1, $2, $3	X"00430820"	X"00000004"	X"0...2"	X"0...3"	X"0...0010"	X"0...5"	X"0...1"	X"0...2800"		
1	SUB $1, $2, $3	X"00430822"	X"00000008"	X"0...2"	X"0...3"	X"0...FFFD"	X"F...F"	X"0...0"	X"F...FFD8"		
2	SLL $1, $2, 1	X"00020840"	X"0000000C"	X"0...0"	X"0...2"	X"0...0400"	X"0...04"	X"0...1"	X"0...2100"		
3	SRL $1, $2, 2	X"00020882"	X"00000010"	X"0...0"	X"0...2"	X"0...0002"	X"0...0"	X"0...0"	X"0...2084"		
4	AND $1, $2, $3	X"00430824"	X"00000014"	X"0...2"	X"0...3"	X"0...0824"	X"0...2"	X"0...0"	X"0...2084"		
5	OR $1, $2, $3	X"00430825"	X"00000018"	X"0...2"	X"0...3"	X"0...0825"	X"0...3"	X"0...0"	X"0...2085"		
6	XOR $1, $2, $3	X"00430826"	X"0000001C"	X"0...2"	X"0...3"	X"0...0826"	X"0...1"	X"0...0"	X"0...2086"		
7	SLT $1, $2, $3	X"0043083A"	X"00000020"	X"0...2"	X"0...3"	X"0...0829"	X"0...1"	X"0...0"	X"0...28"		
8	ADDI $1, $2, 1	X"FC410001"	X"00000024"	X"0...2"	X"0...1"	X"0...01"	X"0...3"	X"0...0"	X"0...30"		
9	LW $1, 2($2)	X"F8410002"	X"00000028"	X"0...2"	X"0...3"	X"0...02"	X"0...4"	X"0...1"	X"0...38"		
10	SW $1, 3($2)	X"F4410003"	X"0000002C"	X"0...2"	X"0...3"	X"0...03"	X"0...5"	X"0...1"	X"0...40"		
11	BEQ $1, $2, 4	X"F0410004"	X"00000030"	X"0...2"	X"0...1"	X"0...04"	X"0...1"	X"0...0"	X"0...48"	X"0...40"	
12	BGEZ $1, 5	X"EC200005"	X"00000034"	X"0...1"	X"0...0"	X"0...05"	X"FFF...6"	X"0...0"	X"0...4C"	X"0...48"	

Restul lucrurilor nu mai intra in program deoarece avem iin felul urmator:
BEQ $1, $2, 4 aici se compara 1 cu 2, dar conditia este falsa deci nu va sari nicaier, se merge mai departe, dar... in momentul in care noi facem BGEZ $1, 5 $1 = 1 deci se indeplineste conditia, se executa branch si iese din progam

Probleme întâmpinate: - lung. lo Pc (rezolvat)
- branch nu funcționau corect inițial (rezolvat)
- dificultăți la UC (rezolvat)

Testare: - testat în simulator
- testat pe placă